

```
// File: src/App.vue
<script setup>
import { RouterView } from 'vue-router'
import ToastContainer from './components/ToastContainer.vue'
import ErrorBoundary from './components/ErrorBoundary.vue'
import PWAUpdatePrompt from './components/PWAUpdatePrompt.vue'
</script>
<template>
  <div class="app-shell">
    <a class="skip-link" href="#main-content">跳到主要内容</a>
    <ErrorBoundary>
      <RouterView />
    </ErrorBoundary>
    <ToastContainer />
    <PWAUpdatePrompt />
  </div>
</template>
<style scoped>
.app-shell {
  min-height: 100vh;
}
.skip-link {
  position: fixed;
  top: max(8px, env(safe-area-inset-top));
  left: 12px;
  z-index: 10000;
  padding: 10px 14px;
  border-radius: var(--radius-sm);
  background: var(--text-primary);
  color: white;
  font-weight: 800;
  transform: translateY(-160%);
  transition: transform var(--duration-fast) var(--ease-out);
}
.skip-link:focus-visible {
  transform: translateY(0);
}
</style>
// File: src/main.js
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import './style.css'
const app = createApp(App)
app.use(router)
app.mount('#app')
// File: src/router.js
import { createRouter, createWebHistory } from 'vue-router'
const routes = [
```

{

```

    path: '/',
    name: 'Home',
    component: () => import('./pages/Home.vue'),
    meta: { title: '首页' }
  },
  {
    path: '/difficulty',
    name: 'DifficultySelect',
    component: () => import('./pages/DifficultySelect.vue'),
    meta: { title: '选择关卡' }
  },
  {
    path: '/game/:id',
    name: 'Game',
    component: () => import('./pages/Game.vue'),
    props: true,
    meta: { title: '闯关' }
  },
  {
    path: '/explore',
    name: 'NumberExplore',
    component: () => import('./pages/NumberExplore.vue'),
    meta: { title: '数字探索' }
  },
  // 404 - 未匹配的路由重定向到首页
  {
    path: '/*',
    redirect: '/'
  }
]

const router = createRouter({
  history: createWebHistory(),
  routes,
  scrollBehavior() {
    return { left: 0, top: 0 }
  }
})

router.afterEach((to) => {
  document.title = `${to.meta.title || '儿童数学启蒙训练'} | 数感闯关`
})

export default router

```

// File: src/pages/DifficultySelect.vue

```

<script setup>
import { computed, nextTick, onMounted, ref } from 'vue'
import { useRouter } from 'vue-router'
import { ArrowLeft, Trophy } from 'lucide-vue-next'
import { DIFFICULTY_GROUPS, getDifficultyById, TOTAL_LEVELS } from '../config/difficulty'
import { useStorage } from '../composables/useStorage'
import { useSound } from '../composables/useSound'

```

```
import DifficultyCard from '../components/DifficultyCard.vue'
```

```
const router = useRouter()
const { getBestScore, completedIds, getLeaderboard } = useStorage()
const { playClick, playBack } = useSound()
const isLeaving = ref(false)
const BACK_NAVIGATION_DELAY = 180
const SCROLL_STORAGE_KEY = 'math-game-difficulty-scroll-y'
const completedCount = computed(() => completedIds.value.length)
const progressPercent = computed(() => Math.round((completedCount.value / TOTAL_LEVELS) * 100))
function getSavedScrollY() {
  try {
    return Number(sessionStorage.getItem(SCROLL_STORAGE_KEY))
  } catch (error) {
    console.error('读取关卡页滚动位置失败:', error)
    return 0
  }
}
function saveScrollY() {
  try {
    sessionStorage.setItem(SCROLL_STORAGE_KEY, String(window.scrollY))
  } catch (error) {
    console.error('保存关卡页滚动位置失败:', error)
  }
}
function clearSavedScrollY() {
  try {
    sessionStorage.removeItem(SCROLL_STORAGE_KEY)
  } catch (error) {
    console.error('清理关卡页滚动位置失败:', error)
  }
}
onMounted(async () => {
  const savedScrollY = getSavedScrollY()
  if (!Number.isFinite(savedScrollY) || savedScrollY <= 0) {
    window.scrollTo(0, 0)
    return
  }
  await nextTick()
  requestAnimationFrame(() => {
    window.scrollTo(0, savedScrollY)
    clearSavedScrollY()
  })
})
function goBack(event) {
  if (isLeaving.value) {
    return
  }
  isLeaving.value = true
  clearSavedScrollY()
  playBack()
}
```

```
const btn = event?.currentTarget
```

```
if (btn) {
  btn.classList.add('is-leaving')
}
setTimeout(() => {
  router.push('/')
}, BACK_NAVIGATION_DELAY)
}
function selectDifficulty(event, difficulty) {
  if (isLeaving.value) {
    return
  }
  isLeaving.value = true
  saveScrollY()
  playClick()
  const card = event?.currentTarget?.closest('.difficulty-card')
  if (card) {
    card.classList.add('is-leaving')
  }
  router.push(`/game/${difficulty.id}`)
}
function getDifficultyBestScore(difficultyId) {
  return getBestScore(difficultyId)
}
function getDifficultyLeaderboard(difficultyId) {
  const difficulty = getDifficultyById(difficultyId)
  return getLeaderboard(difficultyId, difficulty.questionCount)
}
const lockedStatusMap = computed(() => {
  const map = new Map()
  const completed = completedIds.value
  DIFFICULTY_GROUPS.forEach((group) => {
    group.levels.forEach((id) => {
      if (id === 1) {
        map.set(id, false)
        return
      }
      map.set(id, !completed.includes(id - 1))
    })
  })
  return map
})
function isDifficultyLocked(difficulty) {
  return lockedStatusMap.value.get(difficulty.id) ?? false
}
</script>
<template>
  <div class="page">
    <header class="header-panel">
      <div class="nav-row">
```

<button class="back-btn" aria-label="返回首页" @click="goBack(\$event)">

```

        <ArrowLeft :size="18" aria-hidden="true" />
    </button>
    <div class="header-badge">
        <Trophy :size="16" aria-hidden="true" />
        <span>{{ completedCount }}/{{ TOTAL_LEVELS }}</span>
    </div>
</div>
<div class="headline-row">
    <div>
        <p class="eyebrow">选择关卡</p>
        <h1 class="title text-child-2xl">选一关开始</h1>
        <p class="subtitle text-child-sm">答对 85% 就能过关，过关后解锁下一关。</p>
    </div>
</div>
<div class="progress-panel">
    <div class="progress-copy">
        <span>已过关</span>
        <strong>{{ progressPercent }}%</strong>
    </div>
    <div
        class="progress-track"
        role="progressbar"
        aria-label="关卡完成进度"
        aria-valuemin="0"
        :aria-valuemax="TOTAL_LEVELS"
        :aria-valuenow="completedCount"
    >
        <div class="progress-fill" :style="{ width: `${progressPercent}%` }"></div>
    </div>
</div>
</header>
<main id="main-content" class="main-content">
    <section
        v-for="group in DIFFICULTY_GROUPS"
        :key="group.name"
        class="section"
    >
        <div class="section-header">
            <h2 class="section-badge" :class="'badge-${group.color}'">{{ group.name }}</h2>
        </div>
        <div class="card-list">
            <DifficultyCard
                v-for="id in group.levels"
                :key="id"
                :difficulty="getDifficultyById(id)"
                :is-locked="isDifficultyLocked(getDifficultyById(id))"
                :is-completed="completedIds.includes(id)"
                :best-score="getDifficultyBestScore(id)"
                :leaderboard="getDifficultyLeaderboard(id)"
            >

```

@select="selectDifficulty"

```
    />
  </div>
</section>
</main>
</div>
</template>
<style scoped>
.page {
  min-height: 100vh;
  min-height: 100dvh;
  padding: calc(env(safe-area-inset-top, 0px) + 18px) 14px 36px;
}

.header-panel {
  position: sticky;
  top: calc(env(safe-area-inset-top, 0px) + 10px);
  z-index: 10;
  margin-bottom: 18px;
  padding: 18px;
  border-radius: var(--radius-xl);
  background: rgba(255, 255, 255, 0.94);
  border: 1px solid var(--border-light);
  box-shadow: var(--shadow-sm);
}

.nav-row,
.progress-copy,
.section-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.nav-row {
  margin-bottom: 18px;
}

.back-btn,
.header-badge {
  display: inline-flex;
  align-items: center;
  border-radius: 14px;
  font-weight: 700;
}

.back-btn {
  justify-content: center;
  width: 48px;
  height: 48px;
  color: var(--text-secondary);
  background: rgba(255, 255, 255, 0.92);
  border: 1px solid var(--border-light);
}

.header-badge {
```

gap: 8px;

```
}  
.back-btn:active {  
  transform: scale(0.98);  
}  
.back-btn.is-leaving {  
  opacity: 0.7;  
  transform: scale(0.96);  
  transition: opacity var(--duration-normal) var(--ease-out), transform var(--duration-normal) var(--ease-out);  
}  
@media (hover: hover) {  
  .back-btn:hover {  
    color: var(--text-primary);  
    background: rgba(255, 255, 255, 0.9);  
  }  
}  
.header-badge {  
  padding: 10px 12px;  
  color: var(--candy-blue-dark);  
  background: rgba(49, 120, 246, 0.08);  
}  
.headline-row {  
  margin-bottom: 18px;  
}  
.eyebrow {  
  margin-bottom: 8px;  
  color: var(--candy-blue-dark);  
  font-size: var(--font-sm);  
  font-weight: 800;  
  letter-spacing: 0.1em;  
  text-transform: uppercase;  
}  
.title {  
  color: var(--text-primary);  
  font-weight: 800;  
}  
.subtitle {  
  margin-top: 8px;  
  color: var(--text-secondary);  
  line-height: 1.7;  
}  
.progress-panel {  
  padding: 16px;  
  border-radius: var(--radius-lg);  
  background: rgba(255, 255, 255, 0.9);  
  border: 1px solid var(--border-light);  
}  
.progress-copy {  
  margin-bottom: 10px;  
  color: var(--text-secondary);
```

font-size: var(--font-sm);

```
    font-weight: 700;
  }
  .progress-copy strong {
    color: var(--text-primary);
  }
  .progress-track {
    height: 10px;
    overflow: hidden;
    border-radius: var(--radius-full);
    background: rgba(49, 120, 246, 0.12);
  }
  .progress-fill {
    height: 100%;
    border-radius: inherit;
    background: linear-gradient(90deg, var(--candy-blue), var(--candy-mint));
  }
  .main-content {
    display: flex;
    flex-direction: column;
    gap: 18px;
  }
  .section {
    opacity: 1;
    transform: none;
  }
  .section-header {
    margin-bottom: 8px;
    padding: 4px 2px;
  }
  .section-badge {
    display: inline-flex;
    align-items: center;
    width: fit-content;
    border-radius: 0;
    font-weight: 800;
    padding: 0;
    color: var(--text-muted);
    font-size: 12px;
  }
  .badge-mint {
    background: var(--candy-mint-soft);
    color: var(--candy-mint-dark);
  }
  .badge-pink {
    background: var(--candy-blue-soft);
    color: var(--candy-blue-dark);
  }
  .badge-yellow {
    background: var(--candy-yellow-soft);
```

color: var(--candy-yellow-dark);

```
}  
.badge-peach {  
  background: var(--candy-peach-soft);  
  color: var(--candy-peach-dark);  
}  
.badge-lavender {  
  background: var(--candy-lavender-soft);  
  color: var(--candy-lavender-dark);  
}  
.card-list {  
  display: flex;  
  flex-direction: column;  
  gap: 12px;  
}  
@media (min-width: 768px) {  
  .page {  
    max-width: 980px;  
    margin: 0 auto;  
    padding: 24px 20px 40px;  
  }  
  .card-list {  
    display: grid;  
    grid-template-columns: repeat(2, minmax(0, 1fr));  
  }  
}  
@media (max-width: 420px) {  
  .page {  
    padding: 12px 10px 28px;  
  }  
  .header-panel {  
    top: 8px;  
    margin-bottom: 14px;  
    padding: 14px;  
    border-radius: 24px;  
  }  
  .nav-row {  
    margin-bottom: 14px;  
  }  
  .back-btn,  
  .header-badge {  
    padding: 9px 10px;  
  }  
  .title {  
    font-size: 26px;  
    line-height: 1.2;  
  }  
  .subtitle {  
    margin-top: 6px;  
    line-height: 1.55;
```

}

```
.progress-panel {
  padding: 14px;
}

.section-header {
  padding: 2px 2px 0;
}
}
</style>
// File: src/pages/Game.vue
<script setup>
import { computed, nextTick, onMounted, onUnmounted, ref, watch } from 'vue'
import { useRouter } from 'vue-router'
import { AlertCircle, ArrowLeft, CheckCircle2, RotateCcw, Star } from 'lucide-vue-next'
import { getDifficultyById } from '../config/difficulty'
import { GAME_CONFIG, STORAGE_KEYS } from '../config/constants'
import { useGame } from '../composables/useGame'
import { useStorage } from '../composables/useStorage'
import { useToast } from '../composables/useToast'
import { useSound } from '../composables/useSound'
import { getStarCount } from '../utils/stars'
import NumberPad from '../components/NumberPad.vue'
import QuestionCard from '../components/QuestionCard.vue'
import ResultModal from '../components/ResultModal.vue'
import ScoreBoard from '../components/ScoreBoard.vue'
const props = defineProps({
  id: {
    type: String,
    required: true
  }
})
const router = useRouter()
const difficulty = getDifficultyById(parseInt(props.id))
if (!difficulty) {
  router.replace('/difficulty')
}
const { getLeaderboard, updateBestScore } = useStorage()
const { error: showError } = useToast()
const {
  playCorrect,
  playWrong,
  playVictory,
  playUnlock,
  playResultPraise,
  playBack,
  playKeyPress,
  playDelete,
  playSubmit,
  stopPraise
} = useSound()
```

```
const game = useGame(difficulty)
```

```

const showAnswer = ref(false)
const isWaiting = ref(false)
const userAnswer = ref('')
const modalResult = ref(null)
const modalVisible = ref(false)
const isLoading = ref(true)
const currentFeedbackState = ref('idle')
const streakCount = ref(0)
const showStreakReward = ref(false)
const streakRewardText = ref('')
const showNumberBondHint = ref(false)
const errorContinueButton = ref(null)
watch(modalResult, (result, prev) => {
  if (result && !prev) {
    modalVisible.value = true
  } else if (!result && prev) {
    modalVisible.value = false
  }
})
watch(currentFeedbackState, async (state) => {
  if (state !== 'incorrect-manual') return
  await nextTick()
  errorContinueButton.value?.focus()
})
const lastInputTime = ref(0)
const lastSubmitTime = ref(0)
const INPUT_DEBOUNCE = 45
const SUBMIT_DEBOUNCE = 120
let feedbackTimeout = null
let streakRewardTimeout = null
let unlockTimeout = null
let resultPraiseTimeout = null
let retryQuestions = null
const isReviewRound = ref(false)
const isComplete = computed(() => game.isComplete.value)
const currentQuestion = computed(() => game.currentQuestion.value)
const isCorrect = computed(() => currentQuestion.value?.isCorrect === true)
const isIncorrect = computed(() => currentQuestion.value?.isCorrect === false)
const shouldShowFeedback = computed(() => showAnswer.value && currentQuestion.value?.userAnswer !== null)
const currentQuestionHasNumberBondHint = computed(() => {
  const question = currentQuestion.value
  if (!question) return false
  const missingPart = question.missingPart || 'answer'
  let ballCount = 0
  if (question.operator === '+' && missingPart !== 'answer') {
    ballCount = question.result || question.answer || 0
  } else if (question.operator === '+') {
    ballCount = (Number(question.operand1) || 0) + (Number(question.operand2) || 0)
  } else if (question.operator === '-') {

```

```
ballCount = Number(question.operand1) || 0
```

```
}
return ballCount > 0
})
function loadNumberBondHintSetting() {
  try {
    const saved = localStorage.getItem(STORAGE_KEYS.NUMBER_BOND_HINT)
    showNumberBondHint.value = saved === null ? false : saved === 'true'
  } catch (error) {
    console.error('读取小球提示设置失败:', error)
    showNumberBondHint.value = false
  }
}
function toggleNumberBondHint() {
  showNumberBondHint.value = !showNumberBondHint.value
  try {
    localStorage.setItem(STORAGE_KEYS.NUMBER_BOND_HINT, String(showNumberBondHint.value))
  } catch (error) {
    console.error('保存小球提示设置失败:', error)
  }
  playKeyPress()
}
function triggerHapticFeedback(level = 'medium') {
  if (navigator.vibrate) {
    if (level === 'light') {
      navigator.vibrate(20)
      return
    }
    if (level === 'strong') {
      navigator.vibrate([28, 20, 32])
      return
    }
    if (level === 'error') {
      navigator.vibrate([24, 24, 24])
      return
    }
    navigator.vibrate(50)
  }
}
function initGame() {
  isLoading.value = true
  game.startGame(retryQuestions ? { questions: retryQuestions } : undefined)
  retryQuestions = null
  isLoading.value = false
}
function submitAnswer() {
  if (isLoading.value || !userAnswer.value || userAnswer.value.trim() === '') {
    showError('先输入答案')
    return
  }
}
```

```
const now = Date.now()
```

```

if (now - lastSubmitTime.value < SUBMIT_DEBOUNCE) return
lastSubmitTime.value = now
const answer = parseInt(userAnswer.value)
const correct = game.submitAnswer(answer)
showAnswer.value = true
isWaiting.value = true
playSubmit()
if (correct) {
  streakCount.value += 1
  triggerStreakReward()
  triggerHapticFeedback('strong')
  playCorrect()
} else {
  streakCount.value = 0
  showStreakReward.value = false
  triggerHapticFeedback('error')
  playWrong()
}
if (correct) {
  currentFeedbackState.value = 'correct-auto'
  feedbackTimeout = setTimeout(() => {
    handleNextQuestion()
  }, GAME_CONFIG.FEEDBACK_DELAY)
  return
}
currentFeedbackState.value = 'incorrect-manual'
}

function triggerStreakReward() {
  const isFirstReward = streakCount.value === 3
  const isMilestoneReward = streakCount.value > 0 && streakCount.value % 5 === 0
  if (!isFirstReward && !isMilestoneReward) {
    return
  }
  streakRewardText.value = `连对${streakCount.value}题`
  showStreakReward.value = true
  if (streakRewardTimeout) {
    clearTimeout(streakRewardTimeout)
  }
  streakRewardTimeout = setTimeout(() => {
    showStreakReward.value = false
  }, 760)
}

function handleInput(num) {
  if (isWaiting.value) return
  const now = Date.now()
  if (now - lastInputTime.value < INPUT_DEBOUNCE) return
  lastInputTime.value = now
  if (userAnswer.value.length < GAME_CONFIG.MAX_ANSWER_LENGTH) {
    userAnswer.value += num
  }
}

```

playKeyPress()

```

    return
  }
  triggerHapticFeedback('light')
}
function handleDelete() {
  if (isWaiting.value) return
  const now = Date.now()
  if (now - lastInputTime.value < INPUT_DEBOUNCE) return
  lastInputTime.value = now
  userAnswer.value = userAnswer.value.slice(0, -1)
  playDelete()
}
function handleNextQuestion() {
  if (game.currentIndex.value >= game.questions.value.length - 1) {
    showAnswer.value = false
    isWaiting.value = false
    handleGameComplete()
    return
  }
  userAnswer.value = ''
  showAnswer.value = false
  isWaiting.value = false
  currentFeedbackState.value = 'idle'
  game.nextQuestion()
}
function handleFeedbackClick() {
  if (!isCorrect.value && currentFeedbackState.value === 'incorrect-manual') {
    currentFeedbackState.value = 'idle'
    handleNextQuestion()
  }
}
function handleGameComplete() {
  game.completeGame()
  const result = {
    ...game.getResult(),
    isReviewRound: isReviewRound.value
  }
  const updateResult = result.isReviewRound
    ? {
        isNewBest: false,
        leaderboard: getLeaderboard(parseInt(props.id, 10), result.totalCount),
        leaderboardRank: null
      }
    : updateBestScore(parseInt(props.id, 10), result)
  const stars = getStarCount(result.accuracy)
  const didPass = result.accuracy >= GAME_CONFIG.PASS_ACCURACY
  triggerHapticFeedback(stars >= 4 ? 'strong' : 'medium')
  modalResult.value = {
    ...result,

```

isNewBest: updateResult.isNewBest,

```
leaderboard: updateResult.leaderboard || [],
leaderboardRank: updateResult.leaderboardRank
}
if (didPass) {
  playVictory()
  if (updateResult.isNewBest) {
    unlockTimeout = setTimeout(() => playUnlock(), 460)
  }
}
resultPraiseTimeout = setTimeout(() => {
  playResultPraise({
    accuracy: result.accuracy,
    isNewBest: updateResult.isNewBest,
    isReviewRound: result.isReviewRound
  })
}, didPass ? (updateResult.isNewBest ? 920 : 520) : 320)
}
function goBack() {
  playBack()
  router.push('/difficulty')
}
function resetRound({ preserveRetryQuestions = false, reviewRound = false } = {}) {
  stopPraise()
  if (!preserveRetryQuestions) {
    retryQuestions = null
  }
  isReviewRound.value = reviewRound
  modalResult.value = null
  userAnswer.value = ''
  showAnswer.value = false
  isWaiting.value = false
  currentFeedbackState.value = 'idle'
  streakCount.value = 0
  showStreakReward.value = false
  if (feedbackTimeout) {
    clearTimeout(feedbackTimeout)
    feedbackTimeout = null
  }
  if (streakRewardTimeout) {
    clearTimeout(streakRewardTimeout)
    streakRewardTimeout = null
  }
  if (unlockTimeout) {
    clearTimeout(unlockTimeout)
    unlockTimeout = null
  }
  if (resultPraiseTimeout) {
    clearTimeout(resultPraiseTimeout)
    resultPraiseTimeout = null
  }
}
```

}

```
initGame()
}
function handleRetry() {
  resetRound()
}
function handleRetryMistakes() {
  const incorrectQuestions = modalResult.value?.incorrectQuestions || []
  if (!incorrectQuestions.length) {
    handleRetry()
    return
  }
  retryQuestions = incorrectQuestions.map((item) => ({
    operand1: item.operand1,
    operand2: item.operand2,
    operator: item.operator,
    result: item.result,
    missingPart: item.missingPart,
    answer: item.correctAnswer
  }))
  resetRound({
    preserveRetryQuestions: true,
    reviewRound: true
  })
}
function handleHome() {
  stopPraise()
  modalVisible.value = false
  setTimeout(() => {
    modalResult.value = null
    router.push('/difficulty')
  }, 300)
}
function handleKeyPress(event) {
  if (event.key >= '0' && event.key <= '9') {
    handleInput(parseInt(event.key))
  } else if (event.key === 'Backspace') {
    handleDelete()
  } else if (event.key === 'Enter') {
    submitAnswer()
  }
}
onMounted(() => {
  window.scrollTo(0, 0)
  loadNumberBondHintSetting()
  initGame()
  window.addEventListener('keydown', handleKeyPress)
})
onUnmounted(() => {
  window.removeEventListener('keydown', handleKeyPress)
```

stopPraise()

```
if (feedbackTimeout) {
  clearTimeout(feedbackTimeout)
  feedbackTimeout = null
}
if (streakRewardTimeout) {
  clearTimeout(streakRewardTimeout)
  streakRewardTimeout = null
}
if (unlockTimeout) {
  clearTimeout(unlockTimeout)
  unlockTimeout = null
}
if (resultPraiseTimeout) {
  clearTimeout(resultPraiseTimeout)
  resultPraiseTimeout = null
}
})
</script>
<template>
<div v-if="difficulty" class="page">
  <Transition name="fade">
    <div v-if="isLoading" class="loading-overlay" role="status" aria-live="polite">
      <div class="loading-panel">
        <div class="spinner"></div>
        <p class="loading-text text-child-base">准备中...</p>
      </div>
    </div>
  </Transition>
  <header class="header-panel">
    <button class="nav-btn" @click="goBack" aria-label="返回关卡页">
      <ArrowLeft :size="20" aria-hidden="true" />
    </button>
    <div class="title-group">
      <p class="eyebrow">来做题</p>
      <h1 class="title text-child-lg">第{{ difficulty.id }}关</h1>
      <p class="subtitle text-child-sm">{{ difficulty.description }}</p>
    </div>
    <button class="nav-btn nav-btn-accent" title="重新开始" aria-label="重新开始" @click="handleRetry">
      <RotateCcw :size="18" aria-hidden="true" />
    </button>
  </header>
  <main id="main-content" class="main-layout">
    <section class="question-section">
      <Transition name="streak-reward">
        <div v-if="showStreakReward" class="streak-reward">
          <Star :size="16" fill="currentColor" aria-hidden="true" />
          <span>{{ streakRewardText }}</span>
        </div>
      </Transition>
    </section>
  </main>
</div>
```

<div

```

v-if="game.currentQuestion.value"
class="question-shell"
>
<QuestionCard
: class="{ 'is-hidden-on-error': isIncorrect && shouldShowFeedback }"
: question="game.currentQuestion.value"
: difficulty="difficulty"
: show-answer="showAnswer"
: user-answer="userAnswer"
: current-index="game.currentIndex.value"
: total-questions="game.questions.value.length"
: show-number-bond-hint="showNumberBondHint"
: show-number-bond-hint-toggle="currentQuestionHasNumberBondHint"
@toggle-number-bond-hint="toggleNumberBondHint"
/>
</div>
<Transition name="feedback">
<div
v-if="shouldShowFeedback"
class="feedback-wrap"
: class="{ 'is-success': isCorrect, 'is-error': isIncorrect }"
: role="isIncorrect ? 'dialog' : 'status'"
: aria-modal="isIncorrect ? 'true' : undefined"
: aria-labelledby="isIncorrect ? 'incorrect-feedback-title' : undefined"
aria-live="polite"
@keydown.tab.prevent="errorContinueButton?.focus()"
@keydown.esc.prevent="handleFeedbackClick"
>
<!-- 答错时的对比模式内容 -->
<template v-if="isIncorrect">
<div class="error-review-container">
<!-- 这里的 QuestionCard 会显示用户的错误答案 -->
<QuestionCard
: question="game.currentQuestion.value"
: difficulty="difficulty"
: show-answer="true"
: user-answer="userAnswer"
: current-index="game.currentIndex.value"
: total-questions="game.questions.value.length"
: show-number-bond-hint="showNumberBondHint"
class="review-question"
/>
<div class="feedback-card error">
<div class="feedback-icon error">
<AlertCircle :size="22" aria-hidden="true" />
</div>
<p id="incorrect-feedback-title" class="feedback-kicker">正确答案是</p>
<strong class="feedback-main">{{ currentQuestion.answer }}</strong>
<p class="feedback-tap-note">点一下，继续下一题</p>

```

</div>

```

    </div>
    <button
      ref="errorContinueButton"
      class="feedback-continue-action"
      type="button"
      :aria-label="`正确答案是${currentQuestion.answer}, 继续下一题`"
      @click="handleFeedbackClick"
    />
  </template>
  <!-- 答对时的轻量反馈 -->
  <template v-else>
    <div class="feedback-card success">
      <div class="feedback-icon success">
        <CheckCircle2 :size="22" aria-hidden="true" />
      </div>
      <strong class="feedback-main">对啦</strong>
    </div>
  </template>
</div>
</Transition>
</section>
<section class="control-section">
  <div class="score-wrap">
    <ScoreBoard
      :current-index="game.currentIndex.value"
      :total-questions="game.questions.value.length"
      :correct-count="game.correctCount.value"
      :streak="streakCount"
    />
  </div>
  <div class="keypad-wrap">
    <NumberPad
      :disabled="isWaiting || isComplete"
      @input="handleInput"
      @delete="handleDelete"
      @submit="submitAnswer"
    />
  </div>
</section>
</main>
<ResultModal
  v-if="modalResult"
  :show="modalVisible"
  :result="modalResult"
  :is-new-best="modalResult.isNewBest"
  :difficulty-id="props.id"
  :leaderboard="modalResult.leaderboard"
  :leaderboard-rank="modalResult.leaderboardRank"
  @retry="handleRetry"

```

@retry-mistakes="handleRetryMistakes"

```
@home="handleHome"
/>
</div>
</template>
<style scoped>
.page {
  min-height: 100dvh;
  padding: calc(env(safe-area-inset-top, 0px) + 12px) 12px 12px;
}
.header-panel,
.loading-panel {
  background: rgba(255, 255, 255, 0.94);
  border: 1px solid var(--border-light);
  box-shadow: var(--shadow-sm);
}
.header-panel {
  display: grid;
  grid-template-columns: 48px 1fr 48px;
  align-items: center;
  gap: 12px;
  margin-bottom: 12px;
  padding: 12px;
  border-radius: var(--radius-xl);
}
.nav-btn,
.loading-panel {
  display: flex;
  align-items: center;
  justify-content: center;
}
.nav-btn {
  width: 48px;
  height: 48px;
  border: none;
  border-radius: var(--radius-sm);
  background: rgba(255, 255, 255, 0.92);
  color: var(--text-secondary);
}
.nav-btn:active {
  transform: scale(0.97);
}
@media (hover: hover) {
  .nav-btn:hover {
    color: var(--text-primary);
    background: rgba(255, 255, 255, 0.92);
  }
}
.nav-btn-accent {
  color: var(--candy-blue-dark);
```

background: rgba(49, 120, 246, 0.08);

```
}  
.title-group {  
  min-width: 0;  
  text-align: center;  
}  
.eyebrow {  
  margin-bottom: 4px;  
  color: var(--candy-blue-dark);  
  font-size: 12px;  
  font-weight: 800;  
  letter-spacing: 0.1em;  
  text-transform: uppercase;  
}  
.title {  
  color: var(--text-primary);  
  font-weight: 800;  
}  
.subtitle {  
  margin-top: 2px;  
  color: var(--text-secondary);  
  line-height: 1.35;  
}  
.main-layout {  
  display: grid;  
  gap: 10px;  
}  
.question-section,  
.control-section {  
  display: flex;  
  flex-direction: column;  
  gap: 10px;  
}  
.question-section {  
  position: relative;  
  justify-content: flex-start;  
  min-height: 142px;  
}  
.question-shell {  
  position: relative;  
  width: min(100%, 640px);  
  margin: 0 auto;  
}  
.is-hidden-on-error {  
  opacity: 0;  
  visibility: hidden;  
  pointer-events: none;  
}  
.score-wrap {  
  order: 1;
```

}

```
.keypad-wrap {
  order: 2;
}

.feedback-wrap {
  position: absolute;
  inset: 0;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 12px;
  pointer-events: none;
  border-radius: var(--radius-lg);
  z-index: 5;
}

.feedback-wrap.is-success {
  align-items: flex-start;
  padding-top: 12px;
}

.feedback-wrap.is-error {
  position: fixed; /* 全屏拦截 */
  inset: 0;
  z-index: 100;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 16px;
  pointer-events: auto;
  background: rgba(18, 30, 49, 0.45);
  backdrop-filter: blur(6px);
}

.error-review-container {
  position: relative;
  z-index: 1;
  width: 100%;
  max-width: 440px;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 16px; /* 题目和答案之间的间距 */
  animation: feedbackSlideUp 0.4s var(--ease-out);
  pointer-events: none;
}

.feedback-continue-action {
  position: absolute;
  inset: 0;
  z-index: 2;
  width: 100%;
  height: 100%;
  border: 0;
```

background: transparent;

```
    cursor: pointer;
}

.feedback-continue-action:focus-visible {
    outline: none;
}

.feedback-wrap.is-error:focus-within .feedback-card.error {
    box-shadow: 0 12px 32px rgba(255, 107, 107, 0.3), 0 0 0 4px var(--brand-alert-glow);
}

.review-question {
    width: 100% !important;
    background: white !important;
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.2) !important;
    transform: scale(1.02);
}

.feedback-card {
    width: 100%;
    max-width: 360px;
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 8px;
    padding: 20px 18px 18px;
    border-radius: var(--radius-lg);
    background: rgba(255, 255, 255, 0.98);
    border: 1px solid var(--border-default);
    box-shadow: var(--shadow-sm);
}

.feedback-card.success {
    position: absolute;
    top: 12px;
    width: min(100%, 340px);
    flex-direction: row;
    justify-content: center;
    gap: 10px;
    padding: 14px 16px;
    background: rgba(46, 196, 182, 0.1);
    border: 2px solid rgba(78, 205, 196, 0.28);
    box-shadow: var(--shadow-sm);
    pointer-events: none;
}

.feedback-card.error {
    background: white;
    border: 4px solid var(--candy-red-dark);
    box-shadow: 0 12px 32px rgba(255, 107, 107, 0.3);
}

@keyframes feedbackSlideUp {
    from { transform: translateY(20px); opacity: 0; }
    to { transform: translateY(0); opacity: 1; }
}
```

.feedback-icon {

```
width: 36px;
height: 36px;
border-radius: var(--radius-xs);
display: flex;
align-items: center;
justify-content: center;
}
.feedback-icon.success {
color: var(--candy-mint-dark);
background: rgba(78, 205, 196, 0.18);
}
.feedback-icon.error {
color: var(--candy-red-dark);
background: rgba(255, 107, 107, 0.12);
}
.feedback-text {
display: flex;
flex-direction: column;
align-items: flex-start;
gap: 2px;
}
.feedback-kicker {
color: var(--text-primary);
font-size: var(--font-base);
font-weight: 800;
}
.feedback-main {
color: #2D2D2D;
font-size: 32px;
font-weight: 800;
line-height: 1.1;
}
.feedback-card.success .feedback-main {
font-size: 20px;
color: var(--candy-mint-dark);
}
.feedback-card.error .feedback-main {
color: var(--candy-red-dark);
font-size: 52px;
line-height: 1;
}
.feedback-tap-note {
color: var(--text-secondary);
font-size: var(--font-sm);
font-weight: 700;
}
.loading-overlay {
position: fixed;
inset: 0;
```

z-index: 40;

```
display: flex;
align-items: center;
justify-content: center;
padding: 20px;
background: rgba(238, 244, 255, 0.84);
}
.loading-panel {
flex-direction: column;
gap: 14px;
width: 180px;
height: 160px;
border-radius: var(--radius-xl);
}
.spinner {
width: 42px;
height: 42px;
border: 4px solid rgba(49, 120, 246, 0.14);
border-top-color: var(--candy-blue);
border-radius: 50%;
animation: spin 0.8s linear infinite;
}
.loading-text {
color: var(--text-secondary);
font-weight: 700;
}
.feedback-enter-active,
.feedback-leave-active {
transition: opacity var(--duration-normal) var(--ease-out), transform var(--duration-normal) var(--ease-out);
}
.feedback-enter-from,
.feedback-leave-to {
opacity: 0;
transform: translateY(16px);
}
@keyframes spin {
to {
transform: rotate(360deg);
}
}
@media (min-width: 960px) {
.page {
max-width: 1120px;
margin: 0 auto;
padding: 20px;
}
.main-layout {
grid-template-columns: minmax(0, 1.2fr) minmax(320px, 360px);
align-items: start;
}
```

.question-section {

```
    justify-content: center;
    min-height: 500px;
  }
  .control-section {
    position: sticky;
    top: 20px;
  }
}

@media (max-width: 420px) {
  .page {
    padding: 8px;
  }
  .header-panel {
    grid-template-columns: 44px 1fr 44px;
    gap: 8px;
    margin-bottom: 10px;
    padding: 10px;
    border-radius: var(--radius-lg);
  }
  .nav-btn {
    width: 44px;
    height: 44px;
    border-radius: var(--radius-sm);
  }
  .title {
    font-size: 18px;
    line-height: 1.2;
  }
  .eyebrow {
    margin-bottom: 2px;
    font-size: 11px;
  }
  .subtitle {
    margin-top: 2px;
    font-size: 12px;
    line-height: 1.45;
  }
  .main-layout {
    gap: 8px;
  }
  .question-section,
  .control-section {
    gap: 8px;
  }
  .question-section {
    min-height: 124px;
  }
  .feedback-card {
    padding: 16px 14px;
  }
}
```

```
border-radius: var(--radius-lg);
```

```
}
.feedback-card.success {
  padding: 12px 14px;
  border-radius: var(--radius-sm);
}
.streak-reward {
  top: 4px;
  right: 4px;
  padding: 6px 9px;
  font-size: 11px;
}
.feedback-main {
  font-size: 28px;
}
.feedback-card.success .feedback-main {
  font-size: 17px;
}
.feedback-card.error .feedback-main {
  font-size: 44px;
}
}
@media (max-width: 959px) and (max-height: 860px) {
  .header-panel {
    grid-template-columns: 42px 1fr 42px;
    gap: 8px;
    margin-bottom: 8px;
    padding: 8px;
  }
  .nav-btn {
    width: 42px;
    height: 42px;
    border-radius: var(--radius-xs);
  }
  .eyebrow,
  .subtitle {
    display: none;
  }
  .title {
    font-size: 18px;
    line-height: 1.15;
  }
  .main-layout,
  .question-section,
  .control-section {
    gap: 8px;
  }
  .question-section {
    min-height: 112px;
  }
}
```

}

```

@media (max-width: 959px) {
  .score-wrap {
    order: 2;
  }
  .keypad-wrap {
    order: 1;
  }
}
</style>
// File: src/pages/NumberExplore.vue
<script setup>
import { computed, ref } from 'vue'
import { useRouter } from 'vue-router'
import { ArrowLeft, RotateCcw, Target } from 'lucide-vue-next'
import NumberPad from '../components/NumberPad.vue'
import BallArray from '../components/BallArray.vue'
import { useSound } from '../composables/useSound'
import { getResultBallStageClass } from '../utils/exploreStage'
const router = useRouter()
const { playClick, playSubmit } = useSound()
const QUICK_COUNTS = [10, 50, 100, 500, 1000]
const CHALLENGE_RANGES = [
  { label: '1—10', min: 1, max: 10, hint: '先看有几颗，再试着一眼认出来。' },
  { label: '11—19', min: 11, max: 19, hint: '先找一排 10，再数多出来的。' },
  { label: '20—29', min: 20, max: 29, hint: '先看两排 10，再数多出来的。' },
  { label: '30—39', min: 30, max: 39, hint: '先看三排 10，再数多出来的。' },
  { label: '40—49', min: 40, max: 49, hint: '先数整排，再看多出来几颗。' },
  { label: '50—59', min: 50, max: 59, hint: '先找到 50，再看多出来几颗。' },
  { label: '60—69', min: 60, max: 69, hint: '先数有几排，再看零头。' },
  { label: '70—79', min: 70, max: 79, hint: '先找整十，再数多出来的。' },
  { label: '80—89', min: 80, max: 89, hint: '先看有几个十，再看几个一。' },
  { label: '90—99', min: 90, max: 99, hint: '快到 100 了，先找 90 再数零头。' },
  { label: '100—120', min: 100, max: 120, hint: '先找 100，再看多出来多少。' },
  { label: '121—150', min: 121, max: 150, hint: '先看一个百，再数整十和零头。' },
  { label: '151—199', min: 151, max: 199, hint: '先找 100，再看还多几个十和几个一。' },
  { label: '200—300', min: 200, max: 300, hint: '先看有几个百，再看多出来多少。' },
  { label: '301—500', min: 301, max: 500, hint: '先数有几个百，再看整十和零头。' },
  { label: '501—750', min: 501, max: 750, hint: '先看百位，再看十位和个位。' },
  { label: '751—1000', min: 751, max: 1000, hint: '先判断接近哪个整百，再慢慢缩小范围。' }
]
const mode = ref('explore')
const inputNumber = ref('')
const currentCount = ref(0)
const showResult = ref(false)
const isShaking = ref(false)
const statusMessage = ref('输入 1—1000，看看有多少颗小球。')
const statusTone = ref('default')
const challengeTargetCount = ref(0)
const challengeRangeIndex = ref(0)

```

```
const challengeCorrectInRange = ref(0)
```

```

const challengeSolvedCount = ref(0)
const challengeLastGuess = ref(null)
const challengeLastCorrect = ref(false)
const isChallengeMode = computed(() => mode.value === 'challenge')
const currentChallengeRange = computed(() => CHALLENGE_RANGES[challengeRangeIndex.value])
const challengeProgressText = computed(() => `本范围答对 ${challengeCorrectInRange.value} 题`)
const resultBallStageClass = computed(() => getResultBallStageClass(currentCount.value))
const challengeStatusText = computed(() => {
  if (!isChallengeMode.value) return ''
  return `当前范围: ${currentChallengeRange.value.label} · ${challengeProgressText.value}`
})

const numberParts = computed(() => {
  const count = currentCount.value
  const hundreds = Math.floor(count / 100)
  const tens = Math.floor((count % 100) / 10)
  const ones = count % 10
  const parts = []
  if (hundreds > 0) parts.push(`${hundreds} 个百`)
  if (tens > 0) parts.push(`${tens} 个十`)
  if (ones > 0 || parts.length === 0) parts.push(`${ones} 个一`)
  return parts.join(' + ')
})

function setStatus(message, tone = 'default') {
  statusMessage.value = message
  statusTone.value = tone
}

function pulseNumber() {
  isShaking.value = true
  window.setTimeout(() => {
    isShaking.value = false
  }, 280)
}

/**
 * Generates a random integer within a range.
 * @param {number} min – Minimum value.
 * @param {number} max – Maximum value.
 * @returns {number} Random integer.
 */
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min
}

function prepareExploreMode() {
  inputNumber.value = ''
  currentCount.value = 0
  showResult.value = false
  challengeLastGuess.value = null
  setStatus('输入 1—1000, 看看有多少颗小球。', 'default')
}

function prepareChallengeRound(keepInput = false) {

```

```
const range = currentChallengeRange.value
```

```
challengeTargetCount.value = randomInt(range.min, range.max)
showResult.value = false
currentCount.value = 0
challengeLastGuess.value = null
challengeLastCorrect.value = false
if (!keepInput) {
  inputNumber.value = ''
}
setStatus(`挑战范围: ${range.label}。${range.hint}`, 'default')
}
function startChallenge(resetProgress = false) {
  if (resetProgress) {
    challengeRangeIndex.value = 0
    challengeCorrectInRange.value = 0
    challengeSolvedCount.value = 0
  }
  prepareChallengeRound()
}
function selectChallengeRange(index) {
  if (challengeRangeIndex.value === index && !showResult.value) return
  playClick()
  challengeRangeIndex.value = index
  challengeCorrectInRange.value = 0
  prepareChallengeRound()
}
function switchMode(nextMode) {
  if (mode.value === nextMode) return
  playClick()
  mode.value = nextMode
  isShaking.value = false
  if (nextMode === 'challenge') {
    startChallenge(true)
    return
  }
  prepareExploreMode()
}
function handleInput(num) {
  playClick()
  if (inputNumber.value === '' && num === 0) {
    setStatus('请从 1 开始, 不能只输入 0。', 'warning')
    pulseNumber()
    return
  }
  const nextValue = `${inputNumber.value}${num}`
  const parsed = Number.parseInt(nextValue, 10)
  if (Number.isNaN(parsed) || parsed > 1000) {
    setStatus('最大可以输入到 1000。', 'warning')
    pulseNumber()
    return
  }
}
```

```

}

<button class="step-btn" type="button" @click="stepCount(1)">+1</button>
<button class="step-btn" type="button" @click="stepCount(10)">+10</button>
</div>
<div class="quick-counts result-counts" aria-label="常用数字">
  <button
    v-for="num in QUICK_COUNTS"
    :key="`result-${num}`"
    class="quick-chip"
    type="button"
    :aria-pressed="currentCount === num"
    @click="jumpToCount(num)"
  >
    {{ num }}
  </button>
</div>
<button class="play-again-btn" @click="resetExplore">
  <RotateCcw :size="20" aria-hidden="true" />
  <span>重新输入</span>
</button>
</section>
</main>
</Transition>
</div>
</template>
<style scoped>
.explore-page {
  min-height: 100dvh;
  display: flex;
  flex-direction: column;
  padding-top: env(safe-area-inset-top);
  padding-bottom: calc(env(safe-area-inset-bottom) + 12px);
  background:
    radial-gradient(circle at top, rgba(92, 157, 255, 0.12) 0%, rgba(92, 157, 255, 0) 38%),
    linear-gradient(180deg, #f8fbff 0%, #edf4ff 100%);
  overflow: hidden;
}
.top-bar {
  display: flex;
  align-items: center;
  padding: 12px 16px 4px;
}
.back-btn {
  width: 48px;
  height: 48px;
  border: 1px solid rgba(92, 157, 255, 0.12);
  border-radius: 18px;
  background: rgba(255, 255, 255, 0.88);
  box-shadow: 0 12px 28px rgba(49, 120, 246, 0.12);
}

```

```
display: inline-flex;  
align-items: center;
```

```
justify-content: center;
color: var(--brand-primary);
backdrop-filter: blur(12px);
-webkit-backdrop-filter: blur(12px);
transition: transform var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), box-shadow var(--duration-fast) var(--ease-standard);
}
.back-btn:focus-visible,
.play-again-btn:focus-visible,
.quick-chip:focus-visible,
.step-btn:focus-visible,
.mode-btn:focus-visible,
.secondary-btn:focus-visible,
.range-chip:focus-visible {
  outline: none;
  box-shadow: 0 0 0 4px rgba(92, 157, 255, 0.18);
}
.back-btn:active,
.play-again-btn:active {
  transform: scale(0.95);
}
.view-wrapper {
  flex: 1;
  display: flex;
  flex-direction: column;
}
.input-screen,
.result-screen {
  flex: 1;
  min-height: 0;
  display: flex;
  flex-direction: column;
  padding: 8px 14px 0;
}
.input-screen {
  gap: 12px;
}
.result-screen {
  gap: 12px;
  padding-inline: 10px;
  padding-bottom: calc(env(safe-area-inset-bottom) + 12px);
  overflow-y: auto;
  -webkit-overflow-scrolling: touch;
}
.mode-switch {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 10px;
}
```

```
.mode-btn,  
.secondary-btn {
```

```
min-height: 48px;
border: 1px solid rgba(92, 157, 255, 0.14);
border-radius: 18px;
background: rgba(255, 255, 255, 0.84);
color: var(--text-blue);
font-size: 15px;
font-weight: 800;
transition: transform var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard);
}

.mode-btn.is-active {
background: linear-gradient(180deg, #6da8ff 0%, #4b86f3 100%);
border-color: rgba(75, 134, 243, 0.24);
color: #fff;
box-shadow: 0 12px 22px rgba(75, 134, 243, 0.18);
}

.range-selector-shell,
.challenge-workspace {
border: 1px solid rgba(92, 157, 255, 0.12);
border-radius: 30px;
background: rgba(255, 255, 255, 0.9);
box-shadow: 0 16px 30px rgba(58, 87, 152, 0.08);
backdrop-filter: blur(14px);
-webkit-backdrop-filter: blur(14px);
}

.range-selector-shell {
padding: 10px 12px;
}

.range-selector-header {
display: flex;
align-items: center;
justify-content: space-between;
gap: 12px;
margin-bottom: 8px;
}

.range-title-badge {
flex-shrink: 0;
}

.range-selector-copy {
margin: 0;
color: var(--text-blue-light);
font-size: 12px;
font-weight: 700;
text-align: right;
}

.challenge-workspace {
padding: 8px;
display: flex;
flex-direction: column;
```

```
gap: 8px;  
}
```

```
.explore-workspace {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.number-stage,
.result-number-shell,
.play-again-btn,
.step-btn,
.challenge-preview-shell {
  position: relative;
  border: 1px solid rgba(92, 157, 255, 0.12);
  background: rgba(255, 255, 255, 0.88);
  box-shadow: 0 16px 30px rgba(58, 87, 152, 0.08);
  backdrop-filter: blur(14px);
  -webkit-backdrop-filter: blur(14px);
}

.number-stage,
.result-number-shell,
.challenge-preview-shell {
  overflow: hidden;
  border-radius: 34px;
}

.number-stage {
  padding: 10px 12px 12px;
  background: rgba(255, 255, 255, 0.96);
  border: 1px solid rgba(92, 157, 255, 0.14);
  box-shadow: 0 20px 36px rgba(58, 87, 152, 0.1);
}

.number-card {
  width: 100%;
}

.number-card-top,
.number-display {
  display: flex;
  align-items: center;
}

.number-card-top {
  justify-content: flex-start;
  gap: 8px;
  margin-bottom: 8px;
}

.counter-badge {
  padding: 8px 12px;
  border-radius: var(--radius-full);
  background: rgba(92, 157, 255, 0.1);
  border: 1px solid rgba(92, 157, 255, 0.12);
  color: var(--brand-primary);
  font-size: var(--font-sm);
}
```

font-weight: 800;

```
}  
.range-badge {  
  background: rgba(107, 203, 119, 0.12);  
  border-color: rgba(107, 203, 119, 0.18);  
  color: var(--text-teal);  
}  
.number-display {  
  justify-content: center;  
  min-height: 76px;  
}  
.big-number {  
  position: relative;  
  z-index: 1;  
  min-width: clamp(88px, 24vw, 132px);  
  padding: 12px 14px;  
  border-radius: var(--radius-lg);  
  border: 2px solid rgba(92, 157, 255, 0.18);  
  background: linear-gradient(180deg, #ffffff 0%, #f1f7ff 100%);  
  box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.92), 0 10px 20px rgba(92, 157, 255, 0.08);  
  color: var(--text-blue-dark);  
  font-size: clamp(48px, 14vw, 76px);  
  font-weight: 800;  
  line-height: 1;  
  text-align: center;  
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), box-shadow var(--duration-fast) var(--ease-standard);  
}  
.big-number.is-empty {  
  color: var(--text-muted);  
}  
.status-copy,  
.result-copy {  
  margin: 0;  
  text-align: center;  
}  
.status-copy {  
  min-height: 16px;  
  margin-top: 6px;  
  color: var(--text-blue-light);  
  font-size: 12px;  
  font-weight: 700;  
  line-height: 1.25;  
}  
.status-copy.is-warning {  
  color: var(--text-warning);  
}  
.status-copy.is-success {  
  color: var(--text-success);  
}
```

```
.challenge-meta,  
.challenge-summary {
```

```
display: grid;
gap: 6px;
margin-top: 8px;
}
.challenge-meta {
  grid-template-columns: repeat(2, minmax(0, 1fr));
}
.challenge-summary {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}
.range-selector {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 6px;
  margin-top: 2px;
}
.range-chip {
  min-height: 44px;
  padding: 0 12px;
  border: 1px solid rgba(92, 157, 255, 0.16);
  border-radius: 999px;
  background: rgba(255, 255, 255, 0.9);
  color: var(--text-blue);
  font-size: 13px;
  font-weight: 800;
  transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard), color var(--duration-fast) var(--ease-standard);
}
.range-chip.is-active {
  background: linear-gradient(180deg, #6ecb96 0%, #4aac70 100%);
  border-color: rgba(74, 172, 112, 0.22);
  color: #fff;
  box-shadow: 0 10px 18px rgba(74, 172, 112, 0.16);
}
.challenge-card {
  min-height: 44px;
  padding: 6px 4px;
  border: 1px solid rgba(92, 157, 255, 0.12);
  border-radius: 14px;
  background: rgba(255, 255, 255, 0.82);
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 2px;
  text-align: center;
}
.challenge-label {
```

color: var(--text-blue-light);
font-size: 10px;

```
    font-weight: 700;
  }
  .challenge-card strong {
    color: var(--text-blue-dark);
    font-size: 13px;
    font-weight: 900;
  }
  .quick-counts {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: 8px;
    margin-top: 14px;
  }
  .quick-chip {
    min-height: 44px;
    padding: 0 14px;
    border: 1px solid rgba(92, 157, 255, 0.16);
    border-radius: 999px;
    background: rgba(255, 255, 255, 0.9);
    color: var(--brand-primary);
    font-size: 14px;
    font-weight: 800;
    transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard);
  }
  .quick-chip:active,
  .step-btn:active,
  .mode-btn:active,
  .secondary-btn:active,
  .range-chip:active {
    transform: scale(0.96);
  }
  .number-stage.is-shaking .big-number {
    border-color: rgba(255, 107, 107, 0.4);
    animation: toy-shake 0.28s ease;
  }
  .number-stage.is-shaking {
    animation: none;
  }
  .challenge-preview-shell {
    padding: 8px;
    border-radius: 22px;
    background: linear-gradient(180deg, rgba(247, 250, 255, 0.98) 0%, rgba(235, 243, 255, 0.95) 100%);
  }
  .preview-copy {
    display: inline-flex;
    align-items: center;
    gap: 8px;
```

color: var(--text-blue);
font-size: 12px;

```
font-weight: 800;
margin-bottom: 6px;
}
.challenge-ball-shell {
border-radius: 20px;
overflow: hidden;
}
.result-number-shell {
display: flex;
align-items: center;
justify-content: center;
padding: 18px 18px 16px;
background: linear-gradient(180deg, rgba(255, 255, 255, 0.98) 0%, rgba(241, 247, 255, 0.96) 100%);
}
.result-number-block {
width: 100%;
display: flex;
flex-direction: column;
align-items: center;
gap: 10px;
}
.result-badge {
background: rgba(107, 203, 119, 0.12);
border-color: rgba(107, 203, 119, 0.18);
color: var(--text-teal);
}
.result-copy {
color: var(--text-blue-light);
font-size: 15px;
font-weight: 800;
line-height: 1.5;
}
.result-ball-shell {
width: 100%;
max-width: min(100%, calc(100dvh - 320px));
min-height: 236px;
margin: 0 auto;
border-radius: 34px;
overflow: hidden;
}
.result-ball-shell--wide {
aspect-ratio: 16 / 9;
}
.result-ball-shell--landscape {
aspect-ratio: 4 / 3;
}
.result-ball-shell--square {
aspect-ratio: 1 / 1;
}
```

```
.result-tools {
```

```
display: flex;
flex-direction: column;
gap: 12px;
}
.step-controls {
display: grid;
grid-template-columns: repeat(3, minmax(0, 1fr));
gap: 10px;
}
.step-btn {
min-height: 52px;
border-radius: 20px;
background: linear-gradient(180deg, rgba(255, 255, 255, 0.98) 0%, rgba(242, 247, 255, 0.96) 100%);
color: var(--text-blue-dark);
font-size: 18px;
font-weight: 900;
transition: transform var(--duration-fast) var(--ease-standard), border-color var(--duration-fast) var(--ease-standard), background var(--duration-fast) var(--ease-standard);
}
.result-counts {
margin-top: 0;
}
.play-again-btn {
width: 100%;
min-height: 64px;
border-radius: 24px;
padding: 0 20px;
display: inline-flex;
align-items: center;
justify-content: center;
gap: 10px;
color: #fff;
font-size: 22px;
font-weight: 900;
background: linear-gradient(180deg, #62c98a 0%, #48aa70 100%);
border-color: rgba(74, 172, 112, 0.24);
box-shadow: 0 12px 26px rgba(74, 172, 112, 0.2);
}
.secondary-btn {
color: var(--text-blue-light);
}
@media (hover: hover) {
.quick-chip:hover,
.step-btn:hover,
.secondary-btn:hover,
.range-chip:hover:not(.is-active) {
border-color: rgba(92, 157, 255, 0.28);
background: var(--bg-light);
}
}
```

```
.mode-btn:hover:not(.is-active) {  
    background: rgba(255, 255, 255, 0.94);  
}
```

```
}
.play-again-btn:hover {
  box-shadow: 0 16px 30px rgba(74, 172, 112, 0.24);
}
}
.fade-enter-active,
.fade-leave-active {
  transition: opacity 0.35s ease, transform 0.35s ease;
}
.fade-enter-from,
.fade-leave-to {
  opacity: 0;
  transform: scale(0.96) translateY(20px);
}
@media (max-width: 420px) {
  .range-selector-header {
    flex-direction: column;
    align-items: flex-start;
  }
  .range-selector-copy {
    text-align: left;
  }
  .number-stage {
    padding: 8px 10px 10px;
  }
  .number-card-top {
    margin-bottom: 8px;
  }
  .number-display {
    min-height: 66px;
  }
  .big-number {
    min-width: 72px;
    padding: 8px 10px;
  }
  .challenge-meta {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
  .challenge-summary {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
  .challenge-workspace {
    padding: 6px;
    gap: 6px;
  }
  .result-copy {
    font-size: 14px;
  }
  .result-ball-shell {
```

min-height: 212px;

```
}  
.step-controls {  
  gap: 8px;  
}  
.step-btn {  
  min-height: 48px;  
  border-radius: 18px;  
  font-size: 17px;  
}  
}  
@media (max-width: 360px) {  
  .counter-badge {  
    padding: 8px 10px;  
    font-size: 12px;  
  }  
  .number-display {  
    min-height: 92px;  
  }  
  .big-number {  
    font-size: 42px;  
  }  
}  
@media (max-width: 959px) and (max-height: 860px) {  
  .number-stage {  
    padding-top: 8px;  
    padding-bottom: 8px;  
  }  
  .number-display {  
    min-height: 88px;  
  }  
  .result-ball-shell {  
    max-width: min(100%, calc(100dvh - 300px));  
    min-height: 196px;  
  }  
}  
@media (min-width: 768px) {  
  .number-stage {  
    padding: 14px 18px 18px;  
  }  
}  
@media (min-width: 960px) {  
  .result-screen {  
    width: min(100%, 1120px);  
    margin: 0 auto;  
    padding-inline: 24px;  
    display: grid;  
    grid-template-columns: minmax(280px, 360px) minmax(420px, 680px);  
    grid-template-rows: auto 1fr;  
    align-items: start;
```

justify-content: center;

```

    gap: 18px;
  }
  .result-number-shell {
    min-height: 220px;
  }
  .result-ball-shell {
    grid-column: 2;
    grid-row: 1 / span 2;
    max-width: min(100%, 680px);
    min-height: 420px;
    align-self: stretch;
  }
  .result-ball-shell--wide,
  .result-ball-shell--landscape {
    min-height: 0;
    align-self: start;
  }
  .result-tools {
    grid-column: 1;
  }
  .result-counts {
    justify-content: flex-start;
  }
}
</style>
// File: src/components/BallArray.vue
<script setup>
import { computed, nextTick, onMounted, onUnmounted, ref, watch } from 'vue'
import { Move3D, RotateCcw } from 'lucide-vue-next'
const props = defineProps({
  count: {
    type: Number,
    required: true,
    validator: (v) => v >= 1 && v <= 1000
  },
  size: {
    type: String,
    default: 'normal',
    validator: (v) => ['normal', 'compact'].includes(v)
  }
})
const canvasRef = ref(null)
const isLoading = ref(true)
const touchDevice = ref(false)
const isRotationEnabled = ref(false)
const renderError = ref(false)
const isSceneReady = ref(false)
const prefersReducedMotion = window.matchMedia('(prefers-reduced-motion: reduce)').matches
const showRotationControls = computed(() => (

```

```
touchDevice.value && isSceneReady.value && props.size === 'normal' && props.count >= 100
```

```
))
const ariaLabel = computed(() => {
  const hundreds = Math.floor(props.count / 100)
  const tens = Math.floor((props.count % 100) / 10)
  const ones = props.count % 10
  const parts = []
  if (hundreds) parts.push(`${hundreds} 个百`)
  if (tens) parts.push(`${tens} 个十`)
  if (ones || !parts.length) parts.push(`${ones} 个一`)
  return `${props.count} 颗小球，按十进制排列为${parts.join('、')}`
})

let scene = null
let camera = null
let renderer = null
let controls = null
let instancedMesh = null
let animationId = null
let renderId = null
let introStartTime = 0
let resizeObserver = null
let orbitDomElement = null
let controlsChangeHandler = null
let THREE = null
let animationDummy = null
function isTouchDevice() {
  return window.matchMedia('(pointer: coarse)').matches || 'ontouchstart' in window
}

const colors = {
  ballLow: 0x8FD0FF,
  ballMid: 0x78AEFF,
  ballHigh: 0x5B8FF2,
  ballPeak: 0x527FE2
}

function getBallPalette(count) {
  if (count <= 19) {
    return {
      color: colors.ballLow
    }
  }
  if (count <= 99) {
    return {
      color: colors.ballMid
    }
  }
  if (count <= 300) {
    return {
      color: colors.ballHigh
    }
  }
}
```

```
return {
```

```

    color: colors.ballPeak
  }
}

function getGroupedBallColor(count, index) {
  const palette = getBallPalette(count)
  const color = new THREE.Color(palette.color)
  if (count < 20) {
    const lightnessBoost = (index % 5) * 0.018
    color.offsetHSL(0, 0, lightnessBoost)
    return color
  }
  if (count < 100) {
    const tenGroup = Math.floor(index / 10)
    const hueShift = (tenGroup % 2 === 0 ? -1 : 1) * 0.012
    const lightnessShift = (tenGroup % 2 === 0 ? 1 : -1) * 0.025
    color.offsetHSL(hueShift, 0.015, lightnessShift)
    return color
  }
  const hundredGroup = Math.floor(index / 100)
  const tenGroup = Math.floor((index % 100) / 10)
  const alternateHundred = hundredGroup % 2 === 1
  const hueShift = alternateHundred ? 0.022 : -0.012
  const saturationShift = alternateHundred ? -0.012 : 0.012
  const hundredLightnessShift = alternateHundred ? 0.045 : -0.018
  const tenLightnessShift = tenGroup % 2 === 0 ? 0.008 : -0.008
  const lightnessShift = hundredLightnessShift + tenLightnessShift
  color.offsetHSL(hueShift, saturationShift, lightnessShift)
  return color
}

async function loadThree() {
  if (THREE) return
  const threeModule = await import('three')
  THREE = threeModule
}

function cleanupControls() {
  if (orbitDomElement) {
    orbitDomElement.style.touchAction = ""
  }
  if (controls && controlsChangeHandler) {
    controls.removeEventListener('change', controlsChangeHandler)
  }
  controlsChangeHandler = null
  orbitDomElement = null
  if (controls) {
    controls.dispose()
    controls = null
  }
}

function updateControlsInteraction() {

```

```
if (!controls || !orbitDomElement || !THREE) return
```

```
if (!touchDevice.value) {
  controls.enabled = true
  controls.enableRotate = true
  controls.enableZoom = true
  controls.enablePan = true
  controls.touches = {
    ONE: THREE.TOUCH.ROTATE,
    TWO: THREE.TOUCH.DOLLY
  }
  orbitDomElement.style.touchAction = 'none'
  return
}

const enabled = showRotationControls.value && isRotationEnabled.value
controls.enabled = enabled
controls.enableRotate = enabled
controls.enableZoom = false
controls.enablePan = false
controls.touches = {
  ONE: THREE.TOUCH.ROTATE,
  TWO: THREE.TOUCH.NONE
}
orbitDomElement.style.touchAction = enabled ? 'none' : 'pan-y'
}

async function loadOrbitControls() {
  if (controls) return
  await loadThree()
  const { OrbitControls } = await import('three/addons/controls/OrbitControls.js')
  orbitDomElement = renderer.domElement
  controls = new OrbitControls(camera, orbitDomElement)
  controls.enableDamping = true
  controls.dampingFactor = 0.08
  controls.minDistance = 3
  controls.maxDistance = 25
  controls.autoRotate = false
  controlsChangeHandler = () => { scheduleRender() }
  controls.addEventListener('change', controlsChangeHandler)
  controls.autoRotateSpeed = 0.7
  controls.rotateSpeed = 0.6
  controls.zoomSpeed = 0.8
  updateControlsInteraction()
}

function clearContainer(container) {
  while (container.firstChild) {
    container.removeChild(container.firstChild)
  }
}

async function initScene() {
  if (!canvasRef.value) return
  renderError.value = false
```

```
isSceneReady.value = false
```

```
try {
  await loadThree()
  await nextTick()
  const container = canvasRef.value
  if (!container) return
  const width = container.clientWidth || 320
  const height = container.clientHeight || 320
  cleanup()
  cleanupControls()
  scene = new THREE.Scene()
  scene.background = null
  camera = new THREE.PerspectiveCamera(32, width / height, 0.1, 200)
  renderer = new THREE.WebGLRenderer({
    antialias: true,
    alpha: true,
    powerPreference: 'high-performance'
  })
  renderer.setSize(width, height)
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))
  renderer.shadowMap.enabled = false
  renderer.outputColorSpace = THREE.SRGBColorSpace
  renderer.toneMapping = THREE.NoToneMapping
  while (container.firstChild) {
    container.removeChild(container.firstChild)
  }
  container.appendChild(renderer.domElement)
  const ambientLight = new THREE.HemisphereLight(0xF8FBFF, 0xDCEBFF, 1.22)
  scene.add(ambientLight)
  const keyLight = new THREE.DirectionalLight(0xFFFFFFFF, 1.06)
  keyLight.position.set(6, 8, 11)
  scene.add(keyLight)
  const rimLight = new THREE.DirectionalLight(0xA8D7FF, 0.62)
  rimLight.position.set(-7, 5, 8)
  scene.add(rimLight)
  const bounceLight = new THREE.PointLight(0xCBE6FF, 0.6, 32, 2)
  bounceLight.position.set(0, -4, 7)
  scene.add(bounceLight)
  await loadOrbitControls()
  isSceneReady.value = true
  await nextTick()
  syncRendererSize()
  updateControlsInteraction()
  createBalls()
  startAnimation()
  isLoading.value = false
} catch (error) {
  console.error('初始化 3D 小球失败:', error)
  cleanupControls()
  cleanup()
```

renderError.value = true

```

    isLoading.value = false
  }
}

function getLayoutConfig(count) {
  if (count >= 1000) {
    return { radius: 0.16, spacing: 0.4, layerGap: 0.64 }
  }
  if (count >= 500) {
    return { radius: 0.17, spacing: 0.42, layerGap: 0.68 }
  }
  if (count >= 100) {
    return { radius: 0.19, spacing: 0.46, layerGap: 0.72 }
  }
  if (count >= 50) {
    return { radius: 0.22, spacing: 0.56, layerGap: 0.72 }
  }
  return { radius: 0.26, spacing: 0.68, layerGap: 0.82 }
}

function createBallMaterial() {
  return new THREE.MeshStandardMaterial({
    color: 0xFFFFFF,
    roughness: 0.52,
    metalness: 0.02
  })
}

/**
 * Rebuilds the instanced mesh while keeping balls that were already visible
 * at their final scale. This avoids replaying the full entrance animation
 * whenever the explored number changes.
 *
 * @param {number} animationStartIndex – Index of the first newly added ball.
 */
function createBalls(animationStartIndex = 0) {
  if (!scene || !camera || !THREE) return
  if (instancedMesh) {
    scene.remove(instancedMesh)
    instancedMesh.geometry?.dispose()
    instancedMesh.material?.dispose()
    instancedMesh = null
  }

  const count = props.count
  const layoutConfig = getLayoutConfig(count)
  const positions = calculatePositions(count, layoutConfig)
  const bounds = getBounds(positions)
  const sphereGeometry = new THREE.SphereGeometry(layoutConfig.radius, 24, 24)
  instancedMesh = new THREE.InstancedMesh(sphereGeometry, createBallMaterial(), count)
  instancedMesh.castShadow = false
  instancedMesh.receiveShadow = false
  const dummy = new THREE.Object3D()

```

animationDummy = dummy

```

const shouldAnimate = !prefersReducedMotion && animationStartIndex < count
for (let i = 0; i < count; i++) {
  const pos = positions[i]
  const ballColor = getGroupedBallColor(count, i)
  dummy.position.set(pos.x, pos.y, pos.z)
  const scale = shouldAnimate && i >= animationStartIndex ? 0 : 1
  dummy.scale.set(scale, scale, scale)
  dummy.updateMatrix()
  instancedMesh.setMatrixAt(i, dummy.matrix)
  instancedMesh.setColorAt(i, ballColor)
}
instancedMesh.instanceMatrix.needsUpdate = true
if (instancedMesh.instanceColor) {
  instancedMesh.instanceColor.needsUpdate = true
}
instancedMesh.userData = {
  positions: positions,
  animating: shouldAnimate,
  animationStartIndex
}
scene.add(instancedMesh)
fitCameraToBounds(bounds)
introStartTime = performance.now()
}

function fitCameraToBounds(bounds) {
  const center = bounds.center
  const layoutConfig = getLayoutConfig(props.count)
  const padding = layoutConfig.radius * 2.5 + 0.8
  const fovRad = camera.fov * (Math.PI / 180)
  const vFovHalf = fovRad / 2
  const aspect = camera.aspect || 1
  const hasDepth = bounds.depth > 0
  const depthWidth = hasDepth ? bounds.depth * 0.45 : 0
  const depthHeight = hasDepth ? bounds.depth * 0.16 : 0
  const distV = ((bounds.height + depthHeight + padding) / 2) / Math.tan(vFovHalf)
  const distH = ((bounds.width + depthWidth + padding) / 2) / Math.tan(vFovHalf) / aspect
  let baseDistance = Math.max(distV, distH) * 1.1 + (bounds.depth / 2)
  let distance = Math.max(baseDistance, 4.5)
  let heightBoost = Math.max(bounds.height * 0.08, 0.84)
  let lookTargetY = center.y
  if (props.count >= 100) {
    heightBoost = Math.max(bounds.height * 0.1, 1)
  }
  if (props.count >= 500) {
    heightBoost = Math.max(bounds.height * 0.12, 1.24)
    lookTargetY = center.y + bounds.height * 0.02
  }
  if (props.count >= 1000) {
    heightBoost = Math.max(bounds.height * 0.14, 1.42)
  }
}

```

```
lookTargetY = center.y + bounds.height * 0.04
```

```

    }
    const cameraYaw = props.count >= 500 ? Math.PI / 7 : hasDepth ? Math.PI / 10 : 0
    const cameraX = center.x + Math.sin(cameraYaw) * distance
    const cameraZ = center.z + Math.cos(cameraYaw) * distance
    camera.position.set(cameraX, center.y + heightBoost, cameraZ)
    camera.lookAt(center.x, lookTargetY, center.z)
    if (controls) {
      controls.target.set(center.x, lookTargetY, center.z)
      controls.update()
    }
  }
}

function calculatePositions(count, layoutConfig) {
  const positions = []
  const spacing = layoutConfig.spacing
  const rowSpacing = count > 10 ? spacing * 1.12 : spacing
  const layerGap = layoutConfig.layerGap
  const centeredOffset = 4.5
  for (let index = 0; index < count; index++) {
    const ones = index % 10
    const tens = Math.floor(index / 10) % 10
    const hundreds = Math.floor(index / 100)
    positions.push({
      x: (ones - centeredOffset) * spacing,
      y: (centeredOffset - tens) * rowSpacing,
      z: (hundreds - (Math.max(Math.ceil(count / 100), 1) - 1) / 2) * layerGap
    })
  }
  return positions
}

function getBounds(positions) {
  if (positions.length === 0) {
    return {
      width: 0,
      height: 0,
      depth: 0,
      center: { x: 0, y: 0, z: 0 }
    }
  }
  let minX = Infinity
  let maxX = -Infinity
  let minY = Infinity
  let maxY = -Infinity
  let minZ = Infinity
  let maxZ = -Infinity
  positions.forEach((p) => {
    minX = Math.min(minX, p.x)
    maxX = Math.max(maxX, p.x)
    minY = Math.min(minY, p.y)
    maxY = Math.max(maxY, p.y)
  })

```

`minZ = Math.min(minZ, p.z)`

```

    maxZ = Math.max(maxZ, p.z)
  })
  return {
    width: maxX - minX,
    height: maxY - minY,
    depth: maxZ - minZ,
    center: {
      x: (minX + maxX) / 2,
      y: (minY + maxY) / 2,
      z: (minZ + maxZ) / 2
    }
  }
}
function scheduleRender() {
  if (!animationId && !renderId && renderer && scene && camera) {
    renderId = requestAnimationFrame(renderOnce)
  }
}
function renderOnce() {
  renderId = null
  if (renderer && scene && camera) {
    controls?.update()
    renderer.render(scene, camera)
  }
}
function easeOutElastic(t) {
  const c4 = (2 * Math.PI) / 3
  return t === 0 ? 0 : t === 1 ? 1 : Math.pow(2, -10 * t) * Math.sin((t * 10 - 0.75) * c4) + 1
}
function animate(time) {
  if (!renderer || !scene || !camera || !instancedMesh?.userData.animating) {
    animationId = null
    return
  }
  controls?.update()
  const { positions, animationStartIndex } = instancedMesh.userData
  let allDone = true
  const elapsed = time - introStartTime
  for (let i = animationStartIndex; i < positions.length; i++) {
    const delay = (positions.length - 1 - i) * 1.5
    const ballElapsed = Math.max(0, elapsed - delay)
    const duration = 650
    let progress = ballElapsed / duration
    if (progress < 1) {
      allDone = false
    } else {
      progress = 1
    }
  }
  const pos = positions[i]

```

```
const scale = easeOutElastic(progress)
```

```
        animationDummy.position.set(pos.x, pos.y, pos.z)
        animationDummy.scale.set(scale, scale, scale)
        animationDummy.updateMatrix()
        instancedMesh.setMatrixAt(i, animationDummy.matrix)
    }
    instancedMesh.instanceMatrix.needsUpdate = true
    renderer.render(scene, camera)
    if (allDone) {
        instancedMesh.userData.animating = false
        animationId = null
        return
    }
    animationId = requestAnimationFrame(animate)
}

function startAnimation() {
    if (!instancedMesh?.userData.animating || animationId) {
        scheduleRender()
        return
    }
    introStartTime = performance.now()
    animationId = requestAnimationFrame(animate)
}

function cleanup() {
    if (animationId) {
        cancelAnimationFrame(animationId)
        animationId = null
    }
    if (renderId) {
        cancelAnimationFrame(renderId)
        renderId = null
    }
    if (renderer) {
        renderer.dispose()
        if (renderer.domElement.parentNode) {
            renderer.domElement.parentNode.removeChild(renderer.domElement)
        }
        renderer = null
    }
    if (instancedMesh) {
        instancedMesh.geometry?.dispose()
        instancedMesh.material?.dispose()
        instancedMesh = null
    }
    scene = null
    camera = null
    animationDummy = null
    isSceneReady.value = false
}

function syncRendererSize() {
```

```
if (!canvasRef.value || !renderer || !camera) return false
```

```
const width = canvasRef.value.clientWidth
const height = canvasRef.value.clientHeight
if (!width || !height) return false
camera.aspect = width / height
camera.updateProjectionMatrix()
renderer.setSize(width, height)
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))
return true
}

function handleResize() {
  if (!syncRendererSize()) return
  scheduleRender()
}

function toggleRotation() {
  isRotationEnabled.value = !isRotationEnabled.value
  updateControlsInteraction()
  scheduleRender()
}

function resetCameraView() {
  const positions = instancedMesh?.userData.positions
  if (!positions?.length) return
  fitCameraToBounds(getBounds(positions))
  scheduleRender()
}

async function retryRender() {
  isLoading.value = true
  renderError.value = false
  await nextTick()
  await initScene()
}

onMounted(() => {
  touchDevice.value = isTouchDevice()
  nextTick(() => {
    initScene()
    window.addEventListener('resize', handleResize)
    if (window.ResizeObserver && canvasRef.value) {
      resizeObserver = new ResizeObserver(() => {
        handleResize()
      })
      resizeObserver.observe(canvasRef.value)
    }
  })
})

onUnmounted(() => {
  cleanup()
  window.removeEventListener('resize', handleResize)
  resizeObserver?.disconnect()
})

watch(() => props.count, async (nextCount, previousCount) => {
```

```
isRotationEnabled.value = false
```

```

    if (renderError.value) return
    await nextTick()
    if (!scene || !renderer) return
    if (animationId) {
      cancelAnimationFrame(animationId)
      animationId = null
    }
    if (renderId) {
      cancelAnimationFrame(renderId)
      renderId = null
    }
    syncRendererSize()
    const animationStartIndex = nextCount > previousCount ? previousCount : nextCount
    createBalls(animationStartIndex)
    updateControlsInteraction()
    startAnimation()
  })
</script>
<template>
  <div
    class="ball-array"
    :class="{
      'ball-array--compact': size === 'compact',
      'ball-array--has-controls': showRotationControls
    }"
  >
    <div v-if="isLoading" class="loading-indicator" aria-hidden="true">
      <span class="dot"></span>
      <span class="dot"></span>
      <span class="dot"></span>
    </div>
    <div
      v-if="renderError"
      class="render-fallback"
      role="group"
      :aria-label="ariaLabel"
      data-testid="ball-render-fallback"
    >
      <strong>暂时无法显示 3D 小球</strong>
      <span>可以重试显示，或刷新页面后再试。</span>
      <button
        class="render-retry-btn"
        type="button"
        data-testid="ball-render-retry"
        @click="retryRender"
      >
        <RotateCcw :size="16" aria-hidden="true" />
        <span>重试显示</span>
      </button>
    </div>
  </div>

```

</div>

```

<div v-else ref="canvasRef" class="canvas-wrapper" role="img" :aria-label="ariaLabel"></div>
<div v-if="showRotationControls" class="view-controls" aria-label="3D 视角控制">
  <button
    class="view-control-btn view-control-btn--toggle"
    :class="{ 'is-active': isRotationEnabled }"
    type="button"
    data-testid="rotation-toggle"
    :aria-label="isRotationEnabled ? '结束旋转并恢复页面滚动' : '启用小球旋转查看'"
    :aria-pressed="isRotationEnabled"
    @click="toggleRotation"
  >
    <Move3D :size="18" aria-hidden="true" />
    <span>{{ isRotationEnabled ? '完成' : '旋转查看' }}</span>
  </button>
  <button
    v-if="isRotationEnabled"
    class="view-control-btn view-control-btn--icon"
    type="button"
    data-testid="rotation-reset"
    aria-label="重置小球视角"
    @click="resetCameraView"
  >
    <RotateCcw :size="18" aria-hidden="true" />
  </button>
</div>
</div>
</template>
<style scoped>
.ball-array {
  position: relative;
  width: 100%;
  height: 100%;
  min-height: 236px;
  border-radius: var(--radius-lg);
  overflow: hidden;
  background: #f4f8ff;
  border: 1px solid rgba(146, 186, 236, 0.32);
  box-shadow: 0 12px 28px rgba(72, 116, 188, 0.1);
}
.canvas-wrapper {
  position: relative;
  width: 100%;
  height: 100%;
  min-height: 236px;
  border-radius: var(--radius-lg);
  overflow: hidden;
  background: transparent;
}
.canvas-wrapper canvas {

```

display: block;

```
width: 100% !important;
height: 100% !important;
}

.render-fallback {
width: 100%;
height: 100%;
min-height: inherit;
padding: 24px;
display: flex;
flex-direction: column;
align-items: center;
justify-content: center;
gap: 6px;
color: var(--text-blue-light);
text-align: center;
}

.render-fallback strong {
color: var(--text-blue-dark);
font-size: 15px;
}

.render-fallback span {
font-size: 13px;
}

.render-retry-btn {
min-height: 44px;
margin-top: 8px;
padding: 0 14px;
border: 1px solid rgba(92, 157, 255, 0.18);
border-radius: 14px;
background: rgba(255, 255, 255, 0.92);
color: var(--brand-primary);
display: inline-flex;
align-items: center;
justify-content: center;
gap: 7px;
font-size: 13px;
font-weight: 800;
touch-action: manipulation;
transition: background var(--duration-fast) var(--ease-out), border-color var(--duration-fast) var(--ease-out), transform var(--duration-fast) var(--ease-out);
}

.render-retry-btn:active {
transform: scale(0.96);
}

.render-retry-btn:focus-visible,
.view-control-btn:focus-visible {
outline: none;
box-shadow: 0 0 0 4px rgba(92, 157, 255, 0.18);
}
```

```
.ball-array--has-controls .canvas-wrapper {  
  height: calc(100% - 56px);
```

```
    min-height: 0;
  }
.view-controls {
  position: absolute;
  right: 8px;
  bottom: 6px;
  z-index: 2;
  min-height: 44px;
  display: flex;
  align-items: center;
  justify-content: flex-end;
  gap: 8px;
}
.view-control-btn {
  min-height: 44px;
  border: 1px solid rgba(92, 157, 255, 0.18);
  border-radius: 14px;
  background: rgba(255, 255, 255, 0.94);
  color: var(--text-blue-dark);
  box-shadow: 0 6px 16px rgba(72, 116, 188, 0.1);
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 7px;
  font-size: 13px;
  font-weight: 800;
  touch-action: manipulation;
  transition: background var(--duration-fast) var(--ease-out), border-color var(--duration-fast) var(--ease-out), color var(--duration-fast) var(--ease-out);
}
.view-control-btn--toggle {
  padding: 0 13px;
}
.view-control-btn--icon {
  width: 44px;
  padding: 0;
}
.view-control-btn.is-active {
  border-color: rgba(74, 144, 226, 0.28);
  background: var(--brand-primary);
  color: #fff;
}
.ball-array--compact,
.ball-array--compact .canvas-wrapper {
  min-height: 138px;
  border-radius: var(--radius-md);
}
.loading-indicator {
  position: absolute;
```

inset: 0;

display: flex;

```
align-items: center;
justify-content: center;
gap: 8px;
z-index: 1;
pointer-events: none;
}
.loading-indicator .dot {
width: 10px;
height: 10px;
border-radius: 50%;
background: rgba(91, 143, 242, 0.6);
animation: dot-pulse 1.2s ease-in-out infinite;
}
.loading-indicator .dot:nth-child(2) {
animation-delay: 0.2s;
}
.loading-indicator .dot:nth-child(3) {
animation-delay: 0.4s;
}
@keyframes dot-pulse {
0%, 80%, 100% {
transform: scale(0.6);
opacity: 0.4;
}
40% {
transform: scale(1);
opacity: 1;
}
}
@media (prefers-reduced-motion: reduce) {
.loading-indicator .dot {
animation: none;
transform: scale(1);
opacity: 0.6;
}
}
@media (max-width: 420px) {
.ball-array,
.canvas-wrapper {
min-height: 212px;
border-radius: var(--radius-md);
}
.ball-array--compact,
.ball-array--compact .canvas-wrapper {
min-height: 124px;
}
}
@media (max-width: 959px) and (max-height: 860px) {
.ball-array,
```

```
.canvas-wrapper {
```

```
    min-height: 196px;
  }
}
</style>
// File: src/composables/useGame.js
import { ref, computed, toValue } from 'vue'
import { generateQuestions, checkAnswer, prepareQuestion } from '../utils/generator'
/**
 * 游戏核心逻辑 Composable
 * @param {Object|Ref<Object>} difficulty - 难度配置对象，支持响应式
 */
export function useGame(difficulty) {
  // 使用 toValue 获取实际值，支持响应式参数
  const difficultyValue = computed(() => toValue(difficulty))
  // 游戏状态
  const questions = ref([])
  const currentIndex = ref(0)
  const score = ref(0)
  const isComplete = ref(false)
  const correctCount = ref(0)
  const incorrectQuestions = ref([]) // 记录错题
  const startTime = ref(null)
  const endTime = ref(null)
  const currentQuestion = computed(() => questions.value[currentIndex.value])
  // 计算属性
  const progress = computed(() => {
    if (!questions.value.length) return 0
    return ((currentIndex.value) / questions.value.length) * 100
  })
  const accuracy = computed(() => {
    if (!questions.value.length) return 0
    const result = Math.round((correctCount.value / questions.value.length) * 100)
    return isNaN(result) ? 0 : result
  })
  const duration = computed(() => {
    if (!startTime.value) return 0
    const end = endTime.value || Date.now()
    const result = Math.floor((end - startTime.value) / 1000)
    return isNaN(result) ? 0 : result
  })
  const durationMs = computed(() => {
    if (!startTime.value) return 0
    const end = endTime.value || Date.now()
    const result = end - startTime.value
    return isNaN(result) ? 0 : Math.max(0, result)
  })
}
/**
 * 开始游戏
 */
```

```
function startGame(options = {}) {
```

```

const customQuestions = Array.isArray(options.questions) ? options.questions : null
questions.value = customQuestions
  ? customQuestions.map((question, index) => prepareQuestion(question, index))
  : generateQuestions(difficultyValue.value)
currentIndex.value = 0
score.value = 0
isComplete.value = false
correctCount.value = 0
incorrectQuestions.value = [] // 重置错题记录
startTime.value = Date.now()
endTime.value = null
}
/**
 * 提交答案
 * @param {number} answer – 用户输入的答案
 */
function submitAnswer(answer) {
  const question = currentQuestion.value
  if (!question) return
  const isCorrect = checkAnswer(question, answer)
  question.userAnswer = answer
  question.isCorrect = isCorrect
  if (isCorrect) {
    score.value += 10
    correctCount.value++
  } else {
    // 记录错题
    incorrectQuestions.value.push({
      id: question.id,
      operand1: question.operand1,
      operand2: question.operand2,
      operator: question.operator,
      result: question.result,
      missingPart: question.missingPart,
      userAnswer: answer,
      correctAnswer: question.answer,
      answer: question.answer
    })
  }
}
// 不在此处切换到下一题，由外部控制
return isCorrect
}
/**
 * 切换到下一题
 */
function nextQuestion() {
  if (currentIndex.value >= questions.value.length - 1) {
    completeGame()
  } else {

```

```
currentIndex.value++
```

```
    }  
  }  
  /**  
   * 完成游戏  
   */  
  function completeGame() {  
    endTime.value = Date.now()  
    isComplete.value = true  
  }  
  /**  
   * 获取游戏结果  
   */  
  function getResult() {  
    return {  
      score: score.value,  
      correctCount: correctCount.value,  
      totalCount: questions.value.length,  
      accuracy: accuracy.value,  
      duration: duration.value,  
      durationMs: durationMs.value,  
      difficulty: difficultyValue.value,  
      completedAt: new Date().toISOString(),  
      incorrectQuestions: incorrectQuestions.value // 返回错题列表  
    }  
  }  
  return {  
    // 状态  
    questions,  
    currentIndex,  
    score,  
    isComplete,  
    correctCount,  
    incorrectQuestions,  
    startTime,  
    endTime,  
    // 计算属性  
    currentQuestion,  
    progress,  
    accuracy,  
    duration,  
    durationMs,  
    // 方法  
    startGame,  
    submitAnswer,  
    nextQuestion,  
    completeGame,  
    getResult  
  }  
}
```

